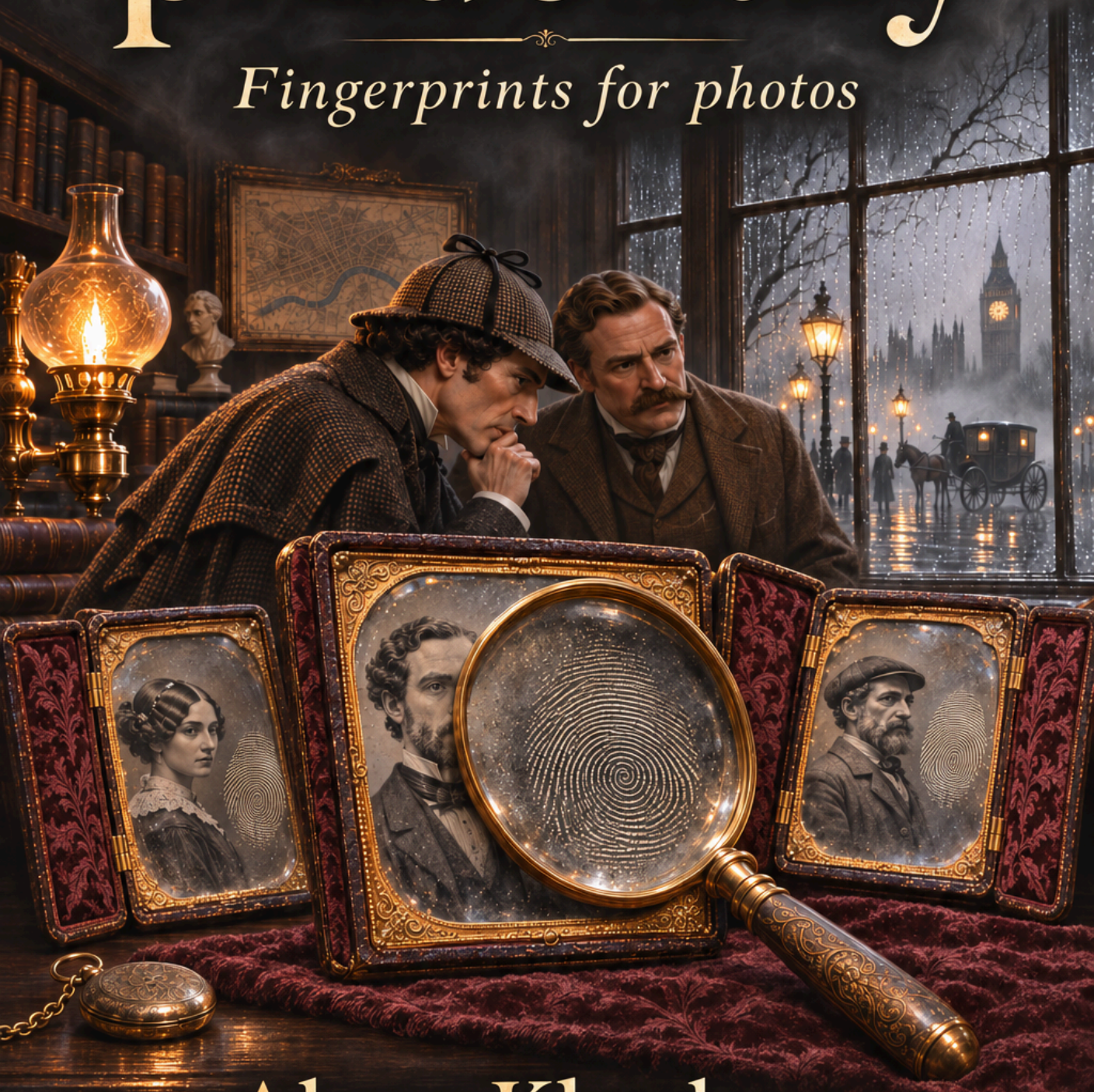
phidentify

Fingerprints for photos



Alexy Khrabrov

FIRST PAIR
PRESS

phidentify

Fingerprints for photos

Alexy Khrabrov

2026-11-02

Contents

About this manual 4
What phidentify is 5
  Where it fits 5
  What it is not 6
Architecture 7
Building 8
  Rust 8
  OCaml and OxCaml 8
  Memory 9
Using it 10
  Rust 10
  OCaml and OxCaml 11
  The phidentify command 12
  The C ABI 12
The identities 14
Whole-file SHA-256 15
  What goes in 15
  What equality proves 15
  What it does not prove 15
  When to use it 15
  Reporting 15
  Cost 15
DNG encoded raw blocks 16
  What goes in 16
  What equality proves 16
  What it does not prove 16
  When to use it 17
  Reporting 17
  Cost 17
Lightroom Classic original fingerprint 18
  What it is 18
  What goes in 18
  Supported strata 18
  Qualification 18
  What equality proves 19
  What it does not prove 19
  When to use it 19
  Reporting 19
  Cost 19
Metadata record 20
  What goes in 20
  What equality proves 20
  What it does not prove 20
  When to use it 20

Compact records and diffs 20
Reporting 21
Cost 21
Licence 21
Preserved digest 22
What it is 22
Where the region comes from 22
The formula 22
Cross-check with Semantic Unison 23
What equality proves 23
What it does not prove 23
When to use it 23
Reporting 23
Cost 24
The identities side by side 25
Which identity answers which question 26
Is this file in my backup? 26
Was this only a metadata edit? 26
Is this still the original my catalog recorded? 26
Did the shuttle copy arrive intact? 26
Testing 27
Licences 28
Reference 29

About this manual

This manual is by Alexy Khrabrov, with Claude as co-author. The ideas and decisions are Alexy's; Claude wrote the text and the code with him.

It describes phidentify 0.1.0 as of 2026-11-02. phidentify is licensed under the MIT License; some optional builds link code under other licences, as the chapter on licences explains. Its source is at github.com/firstpair/phidentify.

Every example in this manual uses synthetic files or generic paths. Nothing here depends on a particular photo library.

What phidentify is

"Are these two photos the same?" has no single answer. Two files can hold the same bytes, or the same encoded raw data with different keywords, or different bytes that Lightroom Classic would still accept as the same original. A whole-file checksum answers the first question and is silent on the others. A metadata comparison answers a fourth question and says nothing about pixels.

phidentify computes several **named identities** for a photo file, each answering one question:

Identity	Rust field	OCaml field	Equal values mean
Whole-file SHA-256	`sha256`	`sha256`	every byte is equal
DNG encoded raw blocks	`dng_raw_blocks_sha256`	`dng_raw_blocks`	the same encoded primary raw data, under the same encoding tags
Lightroom Classic original fingerprint	`lightroom_digest`	`lightroom`	Lightroom Classic's original matcher would pair the files
Metadata record	`metadata_digest`	`metadata`	equal exiftool-rs metadata records, file location left out
Preserved (complement) digest	`preserved_sha256`	`preserved`, with `dng_semantic`	every byte outside the primary XMP packet is equal, at the same size and layout

Three rules hold for every identity, in every interface:

1. **They are separate.** One identity never stands in for another. Equal raw blocks are not equal bytes; an equal Lightroom fingerprint is not equal pixels; an equal metadata record says nothing about image data.
2. **A missing identity is not a match.** An identity that was not requested, does not apply to the file, or failed, is reported as such. Two files that both failed an identity have not matched on it.
3. **Every value comes from one consistent read.** phidentify checks that the file did not change while it was identified. If it did, the whole result is an error.

Where it fits

phidentify is one of three sister projects that grew out of the same photo-archive work. Each owns a different piece.

- **dng-compare** compares DNG archives: it pairs files across two trees, compares their raw samples, decodes when encodings differ, reports metadata differences, and preserves exact older versions as originals or verified deltas. phidentify's raw-block identity is dng-compare's `dng-subset` block comparison turned into a digest, and its metadata identity uses dng-compare's exiftool-rs engine. Its manual is on the First Pair shelf.
- **tiffgraph** is a pure-OCaml, bounded TIFF/DNG graph walker for stock OCaml and OxCaml. Under a fixed grammar it finds a DNG's primary XMP packet, checks that nothing else the file declares lies inside it, and refuses anything outside the grammar with a reason. phidentify uses it to locate the region the preserved digest leaves out. Its manual is on the First Pair shelf.
- **Semantic Unison** is a fork of the Unison file synchronizer that synchronizes DNG metadata separately from image bytes. Peers negotiate which tiffgraph grammar they trust, as the profiles `org.firstpair.dng-xmp/0`, `/1` and `/2`, and Unison owns the `SDE/0` definitions and proofs that a metadata

transfer left every other byte alone. phidentify reproduces Unison's `SDE/0-preserved` hash so that an index built outside Unison speaks the same identity. Its manual is on the First Pair shelf.

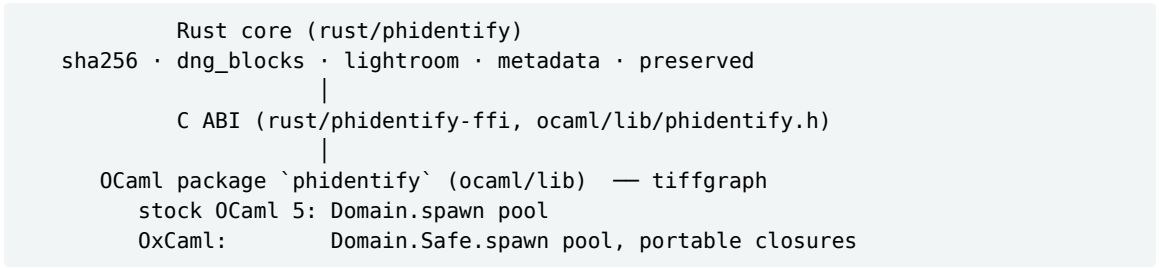
phidentify itself computes identities and nothing else. It does not compare trees, write files, touch catalogs, or synchronize anything.

What it is not

- **Not a rendering identity.** No identity here says that two files look the same after processing. Crop, white balance, profiles and develop settings live in metadata, and the raw-data identities ignore them.
- **Not a decoded-pixel identity**, except inside the Lightroom fingerprint, which decodes for its own purpose and reports only Lightroom's value.
- **Not a validator.** A file that yields every identity is not thereby a correct DNG.
- **Not Lightroom.** The Lightroom fingerprint is an independent reimplementation, qualified against values Lightroom Classic stored. It does not call Adobe code.

Architecture

phidentify is a Rust core with a C ABI, and an OCaml package over that ABI.



The Rust core (`rust/phidentify`, edition 2024, pinned to Rust 1.99.0) holds every identity. `identify(path, Options)` reads the file once and takes every requested identity from the same bytes. The crate forbids `unsafe` code.

The C ABI (`rust/phidentify-ffi`) exposes three functions, declared in `ocaml/lib/phidentify.h`. One call returns every requested identity as a single buffer. It is built as a static library, `libphidentify_ffi.a`, and also as an rlib.

The OCaml package (`ocaml/`, opam name `phidentify`) binds that library and adds two things the Rust core does not do:

- the **tiffgraph walk** that locates a DNG's primary XMP packet under the strictest grammar that accepts it, which supplies the regions for the preserved digest;
- a **domain pool** that identifies many files in parallel and returns results in input order.

The pool is compiled for the switch it is built in. On **stock OCaml 5** it uses `Domain.spawn`. On **OxCaml** it uses `Domain.Safe.spawn`, which accepts only portable closures; the externals it calls carry the `@@ portable` modality, and the paths are shared as an immutable array. dune selects the OxCaml pool when the `stdlib_stable` library exists, which only OxCaml switches provide, and the stock pool otherwise. `Phidentify.backend` reports which one was compiled in: `"domains-stock"` or `"domains-oxcaml"`.

In both pools the tiffgraph walk runs **inside the worker**. Each worker opens the file read-only through a C stub, builds a tiffgraph reader over positional reads on that descriptor, walks, closes, and then calls the Rust identification. Nothing non-portable crosses domains: under OxCaml, tiffgraph's own interface is `@@ portable`, and the reader is built inside the worker rather than captured from outside. The C stubs release the OCaml runtime lock while the Rust core hashes and while the walker's reads run, so the domains really do work in parallel, under stock OCaml and under OxCaml alike.

Building

Rust

The Rust toolchain is pinned in `rust/rust-toolchain.toml` to 1.99.0, with rustfmt and clippy; rustup installs it on first use.

```
git clone https://github.com/firstpair/phidentify.git
cd phidentify/rust
cargo build --release                                  # sha256, DNG raw blocks,
Lightroom
cargo build --release --features phidentify/metadata   # adds the metadata record
cargo test --release
```

The `lightroom` feature is on by default and links rawler 0.8.0 to decode raw samples. The `metadata` feature is off by default; it fetches dng-compare's `dng-exiftool-ffi` crate from GitHub at a pinned commit, and it changes the licence of the resulting binary (see the chapter on licences). Build without default features to get only the whole-file, raw-block and preserved identities:

```
cargo build --release -p phidentify --no-default-features
```

`make test` at the repository root runs the Rust tests, clippy with warnings as errors, and the OCaml tests in the current switch.

OCaml and OxCaml

The OCaml package needs OCaml 5.1 or later (stock OCaml or OxCaml), dune 3.9 or later, and tiffgraph. Its opam file pins tiffgraph to a commit of `firstpair/tiffgraph`; outside opam, pin it yourself:

```
opam pin add tiffgraph git+https://github.com/firstpair/tiffgraph.git#main
```

Build the Rust static library first, then the OCaml package, in the switch you want:

```
(cd rust && cargo build --release -p phidentify-ffi)
cd ocaml
opam exec --switch default  -- dune build && opam exec --switch default  -- dune test
--force
opam exec --switch 5.2.0+ox -- dune build && opam exec --switch 5.2.0+ox -- dune test
--force
```

The switch names are examples: use your own stock OCaml switch and your own OxCaml switch. Test in both if you change anything the pools touch. The test prints the version and the backend, for example `ok phidentify 0.1.0 +lightroom domains-stock` under stock OCaml and `ok phidentify 0.1.0 +lightroom domains-oxcaml` under OxCaml.

To include the metadata identity in the OCaml package, build the FFI library with it: `cargo build --release -p phidentify-ffi --features metadata`. `Phidentify.version ()` then ends in `+metadata`.

Two environment variables control how the Rust library is linked:

- `PHIDENTIFY_FFI_LIBDIR` names the directory holding the Rust artifacts. The default is `../rust/target/release`, relative to the `ocaml/` project.
- `PHIDENTIFY_FFI_LINK` is `static` (the default) or `dynamic`. In static mode the archive is copied into the build tree as a tracked file, so a rebuilt Rust library relinks the executables, and it is installed beside the OCaml library with `phidentify.h`. In dynamic mode the shared library is linked with an rpath to its directory.

Memory

The C library, `phidentify-ffi`, uses mimalloc as the global allocator for Rust allocations. Under the macOS system allocator, metadata extraction through exiftool-rs fragmented the heap and kept about 0.5 MB resident per file in long multi-domain runs; with mimalloc it stays flat. The Rust crate itself leaves the allocator to the program that links it. To measure peak memory one identity at a time, run the probe example: `cargo run --release --example memprobe -- MODE FILE...`, where `MODE` is `sha`, `blocks`, `lightroom`, `metadata` or `all`.

Using it

Rust

```rust
use phidentify::{identify, Options};
use std::path::Path;

let opts = Options { dng_blocks: true, lightroom: true, metadata: false,
preserved_regions: None };
let id = identify(Path::new("photo.dng"), opts)?;
println!("{}", id.sha256);
match &id.dng_raw_blocks_sha256 {
    None => println!("raw blocks: not attempted"),
    Some(Ok(digest)) => println!("raw blocks: {digest}"),
    Some(Err(reason)) => println!("raw blocks failed: {reason}"),
}
```

`Options` has one switch per optional identity:

```rust
pub struct Options {
    pub dng_blocks: bool,
    pub lightroom: bool,
    pub metadata: bool,
    /// Editable regions found by a tiffgraph walk; when given, the
    /// `SDE/0-preserved` complement digest is computed over the same bytes.
    pub preserved_regions: Option<Vec<(u64, u64)>>,
}
```

The whole-file SHA-256 is always computed. `identify` returns `Result<Identity>`:

```rust
pub struct Identity {
    pub stamp: file::Stamp,
    pub sha256: String,
    pub dng_raw_blocks_sha256: Outcome,
    pub lightroom_digest: Outcome,
    pub metadata_digest: Outcome,
    pub preserved_sha256: Outcome,
    pub metadata_json: Option<String>,   // not serialized
}
pub type Outcome = Option<Result<String, String>>;
```

`Outcome` has three states: `None` when the identity was not attempted, `Some(Ok(value))`, and `Some(Err(reason))` when it failed or does not apply. The chapter on reporting lists exactly which happens when.

The Rust core does not walk TIFF graphs for the preserved digest. A Rust caller passes the regions in `Options::preserved_regions`, as sorted, disjoint `(offset, length)` pairs inside the file, obtained from a tiffgraph walk or from Semantic Unison. `phidentify::file::preserved_sha256(bytes, regions)` computes the same digest over bytes already in memory.

The `identify` example prints one JSON line per argument:

```
cargo run --release --example identify -- photo.dng notes.txt
cargo run --release --features phidentify/metadata --example identify -- photo.dng
```

On a synthetic text file, a truncated TIFF and a missing path it prints (shortened with …):

```
{"identity":{"dng_raw_blocks_sha256":null,…,"sha256":"c63c33f0…"},"path":"notes.txt"}
{"identity":{"dng_raw_blocks_sha256":{"Err":"No DNG raw image
found"},…},"path":"tiny.tif"}
{"error":"stat missing: No such file or directory (os error 2)","path":"missing"}
```

In JSON, `null` is not attempted, `{"Ok": …}` is a value and `{"Err": …}` is a failure. The example never passes preserved regions, so `preserved_sha256` is always `null` there.

OCaml and OxCaml

The OCaml interface is the same under stock OCaml and OxCaml.

```
type outcome = Not_attempted | Ok of string | Failed of string

type identity = {
  sha256 : string;
  dng_raw_blocks : outcome;
  lightroom : outcome;
  metadata : outcome;
  dng_semantic : outcome;  (* Ok profile *)
  preserved : outcome;
  metadata_record : string option;
}

type request = { dng_raw_blocks : bool; lightroom : bool; metadata : bool;
                 dng_semantic : bool; compact_record : bool }

val all : request
val version : unit -> string
val backend : string
val recommended_workers : unit -> int
val identify : ?request:request -> string -> (identity, string) result
val identify_all : ?request:request -> workers:int -> string list -> (identity,
string) result list
```

`all` requests every identity and leaves `compact_record` off. A request field that the linked library does not support, such as `metadata` without the `metadata` feature, comes back as `Failed "built without the metadata feature"`, not as `Not_attempted`.

`dng_semantic = true` runs the tiffgraph walk and, when it succeeds, the preserved digest over its region. `dng_semantic` then holds the name of the grammar that accepted the file, and `preserved` the digest.

```
match Phidentify.identify "photo.dng" with
| Error reason -> prerr_endline reason
| Ok id ->
  print_endline id.sha256;
```

```
(match id.dng_semantic, id.preserved with
 | Ok profile, Ok digest -> Printf.printf "%s %s\n" profile digest
 | Failed reason, _ -> Printf.printf "no preserved digest: %s\n" reason
 | _ -> ())
```

`identify_all ~workers paths` identifies a list in parallel and returns the results in input order. It raises `Invalid_argument` when `workers < 1`, and it uses at most `min workers (List.length paths)` domains, capped at `recommended_workers ()`.

The phidentify command

The OCaml package installs `phidentify [-j N] FILE...`. It prints one tab-separated line per file, in argument order:

```
path  sha256  dng_raw_blocks  lightroom  metadata  dng_semantic  preserved
```

Each identity column is the value, `-` when not attempted, or `!reason` when it failed. The command never requests the metadata identity, so that column is always `-`. A file that cannot be identified at all gets a two-column line, `path`, then `!reason`. `-j N` sets the number of workers; the default is `recommended_workers ()`.

On a synthetic text file and a truncated TIFF:

```
notes.txt  c63c33f0…  -                        -  -  !not TIFF            -
tiny.tif   b44eca9f…  !No DNG raw image found  -  -  !missing DNGVersion  -
```

The C ABI

```
#define PHIDENTIFY_DNG_BLOCKS 1u
#define PHIDENTIFY_LIGHTROOM 2u
#define PHIDENTIFY_METADATA 4u
#define PHIDENTIFY_COMPACT_RECORD 8u
int32_t phidentify_identify(const uint8_t *path, size_t path_len, uint32_t flags,
                            const uint64_t *regions, size_t n_regions,
                            uint8_t **out, size_t *out_len);
int32_t phidentify_version(uint8_t **out, size_t *out_len);
void phidentify_free(uint8_t *ptr, size_t len);
```

`path` is a byte string of `path_len` bytes, not NUL-terminated. `regions` holds `n_regions` pairs of `(offset, length)`, or is `NULL` when no preserved digest is wanted.

`phidentify_identify` returns a status:

Status	out holds
0	the identities
1	the reason the file could not be identified
2	nothing; an argument was invalid
3	a message; the Rust core panicked, and the panic was caught

With status 0, `out` holds six fields separated by NUL bytes, in this order: sha256, DNG raw blocks, Lightroom digest, metadata digest, preserved digest, metadata record JSON. Each field is `ok:<value>`, `err:<reason>`, or empty when not attempted. Free every `out` buffer exactly once with `phidentify_free(out, out_len)`.

`phidentify_version` returns a string such as `phidentify 0.1.0 +lightroom +metadata`, naming the features the library was built with.

The identities

The five chapters that follow describe each identity in the same order: what goes in, what equality proves and what it does not, when to use it, how it reports failure and not-applicable, and what it costs.

Whole-file SHA-256

What goes in

Every byte of the file, in order, through SHA-256. The value is 64 lower-case hex digits. It is always computed.

What equality proves

Equal values mean the two files are byte-for-byte identical, up to the collision resistance of SHA-256. Nothing else in phidentify is this strong.

What it does not prove

It does not say anything useful about files that differ. A changed rating, a rewritten preview or one flipped byte of padding all change it, and it cannot tell those apart from a different photograph. File names, dates, permissions and extended attributes are not part of it.

When to use it

Use it when the question is about bytes: is this backup copy exact, did the copy across a network or onto a portable drive arrive intact, is this the file I recorded last year.

Reporting

There is no not-applicable case. If the SHA-256 cannot be computed, the file cannot be identified at all, and the whole call fails: the Rust `identify` returns `Err`, the OCaml functions return `Error reason`, the C ABI returns status 1, and the command prints `path`, then `!reason`. The call also fails if the path is not a regular file (symbolic links are refused) or if the file changes while it is read.

The change check uses a **stamp** taken before and after identification: size, modification time, change time and inode. The device number is recorded but left out of the comparison, because it changes when a volume is remounted. The stamp is returned in the Rust `Identity`.

Cost

One sequential read of the file and one SHA-256 pass. Files up to 1 GiB are read once into memory, and every other identity is computed from those same bytes. Larger files are hashed in 4 MiB chunks and get only this identity.

DNG encoded raw blocks

What goes in

The encoded primary raw images of a DNG, exactly as stored, without decoding. phidentify walks the TIFF or BigTIFF graph: the chain of directories from the header and every SubIFD. A directory is a **primary raw image** when its NewSubfileType is 0 (or absent) and its photometric interpretation is WhiteIsZero, BlackIsZero, RGB, CFA or LinearRaw. The file must also carry a DNGVersion tag; a TIFF without one is not a DNG.

For each primary raw image, phidentify takes:

- its **encoding-tag signature**: the byte order, then, for each tag present from a fixed list of 30 encoding tags, its number, type, count, length and stored value bytes. The list covers dimensions, bits and sample format, compression, photometric interpretation, strip and tile geometry, the predictor, and the JPEG tables and parameters, plus a few DNG encoding tags; it is the `ENCODING` list in `dng_blocks.rs`;
- its **block counts**: how many strips or tiles, and the byte count of each;
- its **block bytes**: the content of every strip or tile, in order.

The digest is SHA-256 over:

```
"PHIDENTIFY-DNG-ENCODED-RAW-BLOCKS-1\0"
‖ byte-order byte (1 for big-endian)
‖ u64le number of primary raw images
‖ for each image, in walk order:
     u64le signature length ‖ signature
   ‖ u64le block count
   ‖ for each block: u64le byte count ‖ block bytes
```

The domain string names the definition. If what it covers ever changes, the domain string changes with it, because every stored digest would change.

What equality proves

Two DNGs have equal raw-block digests exactly when dng-compare's `dng-subset` block comparison finds them equal: the same byte order, the same number of primary raw images, and for each image the same encoding-tag signature, block counts and block bytes. The stored, encoded raw data is the same.

What it does not prove

- **Not metadata equality.** XMP, keywords, ratings, develop settings, Exif fields, maker notes and previews do not enter it. Neither do interpretation tags outside the encoding list, such as black and white levels, the CFA pattern, linearization tables, color matrices and the default crop.
- **Not placement.** Block offsets are left out, so a file whose blocks moved when its metadata grew keeps the same digest. That is the point.
- **Not decoded-pixel equality.** The same samples re-encoded with different compression, tiling or byte order give a different digest. To compare decoded samples across encodings, use dng-compare, which decodes when it has to.
- **Not rendering equality**, and not Lightroom's fingerprint.

When to use it

Use it to recognize the same raw data under changed metadata: a DNG whose keywords were written into the file, a copy whose preview was refreshed, a backup taken before a rating changed. A raw-block match tells you the photograph's encoded data is present in both files. It does not tell you either file is a complete backup of the other.

Reporting

Situation	Rust	OCaml	Command
not requested	`None`	`Not_attempted`	`-`
file does not start with a TIFF header	`None`	`Not_attempted`	`-`
file over 1 GiB	`None`	`Not_attempted`	`-`
TIFF without DNGVersion or without a primary raw image	`Some(Err("No DNG raw image found"))`	`Failed`	`!No DNG raw image found`
malformed graph, offsets or blocks outside the file	`Some(Err(reason))`	`Failed reason`	`!reason`
success	`Some(Ok(hex))`	`Ok hex`	the digest

The walk is bounded: at most 4,096 directories, 8,192 entries per directory, 16 MiB per selected tag value and a million integers per array. A file beyond those bounds fails; it is never truncated into a match.

Cost

Cheap. The walk reads only the directories and the selected tags, and the hash reads the raw blocks once, from the bytes already in memory. There is no decoding. For a typical DNG the cost is close to one SHA-256 pass over the raw data.

Lightroom Classic original fingerprint

What it is

Lightroom Classic stores a fingerprint of each original file in its catalog, in the `digest` column of the `AgLibraryFileDigest` table. phidentify computes that value for the DNGs it supports, so that a file can be checked against the catalog without Lightroom.

It is an **independent reimplementation**. It contains no Adobe code, and it was qualified only by comparing its output with values Lightroom Classic itself had stored.

What goes in

At the level the source shows, the fingerprint has two layers:

1. **A raw-image digest.** The raw samples are decoded with rawler, before linearization, without camera crop or any rendering. The digest is Adobe's NewRawImageDigest, the raw-image digest the DNG specification defines, recomputed here from the decoded samples: the samples are cut into tiles, each tile is hashed with MD5, and the tile digests are hashed again with MD5.
2. **A wrapper.** That raw digest is wrapped, again with MD5, together with streams that describe how the image is framed: the camera model, the default crop size and origin, the user crop when there is one, and a fingerprint stream of the raw image's opcode lists.

The result is 32 upper-case hex digits, as Classic stores it.

The Rust module also exposes `lightroom::compute(path)`, which returns the intermediate values: the raw-image digest, a SHA-256 of the wrapper inputs, and the engine revision name, `native-original-digest-v5`. Those are for diagnosis; the identity is the final digest.

Supported strata

The fingerprint is computed only for DNGs that are:

- integer-sampled,
- single-plane CFA or LinearRaw,
- stored uncompressed or with lossless JPEG compression, in classic TIFF rather than BigTIFF,
- with raw-image opcodes limited to the two implemented, FixVignetteRadial and FixBadPixelsConstant.

Every other file is an error with a reason, never a guessed value. JPEG files, camera-native raw files and DNG strata outside that list are not implemented.

Qualification

On 2026-11-01, 60 randomly chosen DNGs from a Lightroom Classic 15 catalog were fingerprinted. All 60 matched the fingerprints Lightroom Classic had stored for them.

That is the whole evidence for this identity. It covers the strata above, as Lightroom Classic 15 computed them. Extending the fingerprint to another format needs native values from Lightroom for that format first.

What equality proves

An equal digest means Lightroom Classic's original matcher would pair the two files, or would accept the file as the original a catalog entry recorded. Compare it together with the file size, which Classic stores beside it.

What it does not prove

It is native-matcher correspondence, not byte equality and not pixel equality. Two files with equal fingerprints can differ in metadata, previews and layout. MD5 is used because that is what Lightroom uses; the value is a compatibility identity, not a cryptographic one, and should not be relied on against deliberately crafted files.

When to use it

Use it to check a file against a Lightroom Classic catalog: is this the original the catalog thinks it is, which file on disk does this catalog entry correspond to, did a restore bring back the file Classic expects. Read the catalog's stored values from a copy, never from a catalog Classic has open, and never write them.

Reporting

The fingerprint is attempted only when it is requested and the file parsed as a DNG with a primary raw image, the same gate as the raw-block identity. Otherwise it is not attempted (`None`, `Not_attempted`, `-`).

When attempted, it fails with a reason for any unsupported stratum, malformed metadata, a decoder error, decoded dimensions that disagree with the raw directory, or a change to the file while it is read. A library built without the `lightroom` feature reports `built without the lightroom feature` as a failure for every DNG.

Cost

The most expensive identity by far. It reads the file again from its path and fully decodes the raw image, so it costs CPU in proportion to the sensor size and holds the decoded samples in memory, two bytes per sample. Request it only when the question is about Lightroom.

Metadata record

What goes in

The file's metadata as read by dng-compare's **exiftool-rs engine** (`dng-exiftool-ffi`), an in-process Rust metadata reader. The record is one ExifTool-shaped JSON object, following the flags dng-compare's Perl workers use: group-qualified keys (`-G0:1:4`), numeric values (`-n`), nested XMP structures (`-struct`), duplicates kept as `Copy1`, `Copy2` and so on, and unknown tags included. Embedded image payloads such as previews and thumbnails stored as image tags are left out of the record.

Every record carries an engine stamp member, `ExifTool:DngCompareEngine`, naming the engine and its version, for example `exiftool-rs 0.12.0 (ExifTool … tables)`.

The **metadata digest** is SHA-256 over the record serialized with sorted keys, after removing the members that describe where the file sits rather than what it contains: `SourceFile` and every `File:System:*` member (file name, directory, size, dates, permissions).

The engine stamp is **not** removed. It is part of the digest, so records from different engines, or from different versions of the engine, never compare equal. They are separate identities, and records from Perl ExifTool are another identity again.

What equality proves

Equal digests mean the engine produced the same metadata record for both files, apart from location members.

What it does not prove

Nothing about image data: two different photographs can share a record if the engine sees the same fields. Nor does it cover every byte of metadata: fields the engine does not decode, padding, and the bytes of embedded previews are outside it. It is a description of the metadata the engine exposes, not a byte identity of the metadata. To prove that only the metadata packet changed, use the preserved digest.

When to use it

Use it to group files by metadata, to detect metadata-only edits, and, with `diff`, to say exactly which fields changed.

Compact records and diffs

The full record is available too: `metadata_json` in Rust, `metadata_record` in OCaml.

- `metadata::compact(json)` replaces every string member longer than 1 KiB, such as base64 thumbnails and camera-profile tables, by `long-sha256:<hex>:<length>`, the SHA-256 and byte length of the original string. Compact records are for storage. The digest is always taken over the full record, never the compact one. In OCaml, set `compact_record = true` in the request; over the C ABI, pass `PHIDENTIFY_COMPACT_RECORD`.
- `metadata::diff(left, right)` lists the top-level members whose values differ between two records, ignoring location members. Each difference is `(key, left value, right value)`, with `None` on a side where the member is absent. Nested structures are compared as whole values.

Reporting

Situation	Rust	OCaml	Command
not requested	`None`	`Not_attempted`	always -
built without the `metadata` feature	`Some(Err("built without the metadata feature"))`	`Failed`	—
the engine reports an error	`Some(Err("metadata extraction failed: …"))`	`Failed`	—
success	`Some(Ok(hex))`	`Ok hex`	—

The engine reports a failed file as an `ExifTool:Error` member in its record. phidentify turns that member into a failure. It is never treated as an empty record, and two failed files never share a digest.

The metadata identity is attempted for any file, not only DNGs.

Cost

Moderate. The engine reads the file again from its path and decodes its metadata structures; the record can be large when it holds long strings, which is why `compact` exists. It does not decode image data.

Licence

The `metadata` feature links exiftool-rs, which is GPL-3.0-or-later. A binary built with it is covered by the GPL. See the chapter on licences.

Preserved digest

What it is

The preserved digest, also called the complement digest, is a hash of every byte of a DNG **outside** its primary XMP packet, with the file's size and the packet's position bound in. Two files with equal preserved digests can differ only inside that packet. It is Semantic Unison's `SDE/0-preserved` hash, which Unison uses to prove that a metadata transfer changed nothing else.

Where the region comes from

The region left out is found by tiffgraph. phidentify's OCaml package walks each file with three tiffgraph grammars in order, from strictest to most permissive:

Profile reported in `dng_semantic`	tiffgraph grammar	Semantic Unison profile
`dng-xmp/0`	`v0`	`org.firstpair.dng-xmp/0`
`dng-xmp/1`	`v1`	`org.firstpair.dng-xmp/1`
`dng-xmp/2`	`merge_only`	`org.firstpair.dng-xmp/2`

The first grammar that accepts the file wins, and its name is recorded in `dng_semantic`. Accepting the file means tiffgraph found the primary XMP packet and checked that no other extent the file declares lies inside it. The maximum packet size accepted is 1 MiB.

The tiffgraph manual describes what each grammar admits. In short, `v0` admits classic TIFF with standard fields; `v1` adds BigTIFF, embedded camera profiles and the DNG 1.7 JPEG XL fields; `merge_only` adds the private fields camera converters write, kept byte-exact and never interpreted.

The formula

With every integer an unsigned 64-bit big-endian number:

```
SHA-256( "SDE/0-preserved\0"
       ‖ u64be size
       ‖ u64be n
       ‖ (u64be off ‖ u64be len)*          -- each of the n regions
       ‖ for each gap: u64be start ‖ u64be len ‖ bytes )
```

`size` is the file size and `n` the number of regions; phidentify passes one region, the packet. The gaps are the stretches before, between and after the regions, so there are always `n + 1` of them, and each is hashed even when it is empty. That binds the packet's position as well as everything around it.

A small worked example, from the test suite: the 22-byte string `HEADER<xmp one>TRAILER` with one region at offset 6, length 9 hashes

```
"SDE/0-preserved\0" ‖ 22 ‖ 1 ‖ 6 ‖ 9 ‖ 0 ‖ 6 ‖ "HEADER" ‖ 15 ‖ 7 ‖ "TRAILER"
```

Replacing bytes inside the region leaves the digest unchanged; changing any byte outside it changes the digest. Regions must be sorted, disjoint and inside the file, or the digest fails.

Cross-check with Semantic Unison

phidentify's implementation was cross-checked byte for byte against Semantic Unison's own `SemanticDelta.inspect` on 60 DNGs: for every file both accepted, both chose the same strictest grammar and produced the same digest, and files without a packet were refused by both.

What equality proves

Equal preserved digests, **under the same profile**, mean that every byte outside the primary XMP packet is equal, and that the packet sits at the same offset with the same length. The files can differ only inside the packet.

Digests are comparable only under the same profile. The digest depends on the grammar through the region it returns, and a file accepted by `dng-xmp/2` that a stricter peer refuses has no `dng-xmp/0` digest at all. Compare `dng_semantic` before comparing `preserved`.

What it does not prove

Anything about the XMP packet. Two files with equal preserved digests can have completely different keywords, ratings and develop settings. It also does not cover a file whose packet changed size: a longer packet moves the gaps and changes the digest, even if nothing else changed. That is deliberate. Semantic Unison treats such a rewrite as a different operation.

When to use it

Use it to prove a metadata-only edit: the working copy and the backup differ, and you want to know whether only the XMP packet changed. With equal preserved digests, the backup plus the working copy's packet is the working copy. This is the identity a synchronizer needs before it sends metadata without image bytes.

Reporting

Situation	`dng_semantic`	`preserved`	Command
not requested	`Not_attempted`	`Not_attempted`	`-`, `-`
no grammar accepts the file	`Failed reason`	`Not_attempted`	`!reason`, `-`
accepted	`Ok "dng-xmp/N"`	`Ok hex`	`dng-xmp/N`, digest
accepted, file over 1 GiB	`Ok "dng-xmp/N"`	`Failed "file over the in-memory limit"`	profile, `!…`

When every grammar refuses, the reason reported is the last one, from `merge_only`. Typical reasons are `not TIFF`, `missing DNGVersion` and `no XMP packet`. A refused file has **no** preserved digest; it is not a match with anything.

From Rust, the preserved digest is attempted only when `preserved_regions` is given: `None` otherwise, `Some(Err)` for invalid regions or a file over the in-memory limit.

The walk reads the file through its own descriptor before the Rust core reads it for hashing. The Rust stamp check covers the hashing read; a caller that cannot rule out writes between the two should compare stamps or sizes itself.

Cost

A tiffgraph walk, which reads only directories and selected values and is bounded by tiffgraph's limits, then one SHA-256 pass over the file from the bytes already in memory. About the cost of the whole-file SHA-256.

The identities side by side

	SHA-256	DNG raw blocks	Lightroom fingerprint	Metadata record	Preserved
Input	every byte	encoding tags, block counts and bytes of each primary raw image	decoded samples, camera model, crops, opcode lists	exiftool-rs record without location members, with engine stamp	every byte outside the primary XMP packet, with size and layout
Equality proves	identical files	same encoded raw data	Classic would pair the files	same exposed metadata, same engine	only the XMP packet can differ
Does not prove	—	metadata, interpretation tags, decoded or rendered equality	byte or pixel equality	anything about image data or unexposed bytes	anything about the XMP packet
Applies to	every regular file	DNG, ≤ 1 GiB	supported DNG strata	any file the engine reads	DNG accepted by a tiffgraph grammar
Hash	SHA-256	SHA-256	MD5 (Lightroom's)	SHA-256	SHA-256
Cost	one read	parse + raw bytes	full raw decode	metadata parse	walk + one pass
Feature	always	always	`lightroom` (default)	`metadata` (GPL)	always; regions from tiffgraph

Which identity answers which question

Is this file in my backup?

Start with **SHA-256**. If the backup holds a file with the same SHA-256, that file is an exact copy, and nothing more needs to be said.

If it does not, ask why. Compare **DNG raw blocks**: an equal raw-block digest means the backup holds the same encoded photograph, but with different bytes somewhere else. That is not yet a backup of this file. Next compare **preserved** digests under the same profile. If they are equal, the only difference is the XMP packet, and the backup plus this file's packet restores this file exactly. If they differ too, something outside the packet changed, perhaps a refreshed preview or a field a converter rewrote, and the file needs its own copy, or a verified delta such as dng-compare makes.

A failed identity at any step means "not established". It never means "contained".

Was this only a metadata edit?

Compare the two versions:

- SHA-256 differs, **preserved** equal under the same profile: only the XMP packet changed.
- The **metadata** digests then differ, and `metadata::diff` names the fields: a rating, a keyword, a develop setting.
- **preserved** differs but **raw blocks** are equal: the raw data is the same, but something besides the packet changed. Look at the layout, the previews and the private fields before calling it metadata-only.
- **raw blocks** differ: the encoded raw data changed. It may be a re-encode of the same samples, which dng-compare can decide by decoding, or a different photograph.

Is this still the original my catalog recorded?

Compute the **Lightroom fingerprint** and compare it, with the file size, against the `AgLibraryFileDigest.digest` value stored for that original, read from a copy of the catalog. Equal means Lightroom Classic's matcher would accept the file. A file outside the supported strata gives no answer from phidentify; it does not give a "no".

SHA-256 cannot answer this question, because the catalog does not store one, and because Classic's own value deliberately survives metadata written into the file.

Did the shuttle copy arrive intact?

When files travel on a portable drive or across a network and come back, the question is about bytes, so the answer is **SHA-256**: record it before the copy, recompute it after, and compare. Nothing weaker will do, because a damaged byte inside a raw block is still damage even if the metadata looks the same.

If an application on the other side wrote metadata into the files on arrival, SHA-256 will differ. The **preserved** digest then tells you whether the image data and layout arrived intact and only the packet changed, and the **metadata** diff tells you what was written.

Testing

The tests use only synthetic files, built in memory or written to a temporary directory.

- The Rust tests (31 unit tests and 4 integration tests) cover the raw-block identity ignoring XMP and placement but following every raw byte; a non-DNG TIFF and a truncated raw block reported as errors, not matches; each identity reported separately by `identify`; the preserved digest reproducing Unison's formula byte by byte and ignoring region contents; and, for the Lightroom fingerprint, crop and opcode parsing, the opcode fingerprint stream, the tiled digest and the refusal of unsupported strata.
- The OCaml test, run in both switches, checks a non-DNG file (raw blocks and Lightroom not attempted, `dng_semantic` failed, `preserved` not attempted), a missing file reported as an error, and the pool returning forty results in input order with four workers.

```
make test
cd ocaml && opam exec --switch default -- dune test --force
cd ocaml && opam exec --switch 5.2.0+ox -- dune test --force
```

Synthetic tests check the code against its definitions. They are not the qualification of the Lightroom fingerprint, which rests on the native comparison described in its chapter, and they are not a survey of real camera files.

Licences

The phidentify source is MIT. Builds link third-party code under other licences:

- The default `lightroom` feature links rawler 0.8.0, under LGPL-2.1.
- The optional `metadata` feature links exiftool-rs through dng-compare's `dng-exiftool-ffi`, under GPL-3.0-or-later. A binary built with it is covered by the GPL. dng-compare's `rust-scanner/exiftool-ffi/THIRD-PARTY.md` lists the notices.

A build with neither feature links only MIT and permissively licensed crates.

Reference

Item	Value
Version	0.1.0
Rust	1.99.0, edition 2024, `unsafe` forbidden in the core
OCaml	5.1 or later, stock OCaml (`Domain.spawn`) or OxCaml (`Domain.Safe.spawn`)
In-memory limit	1 GiB; larger files get SHA-256 only
Raw-block domain	`PHIDENTIFY-DNG-ENCODED-RAW-BLOCKS-1\0`
Preserved domain	`SDE/0-preserved\0`
Lightroom engine revision	`native-original-digest-v5`
Maximum XMP packet for the walk	1 MiB
C ABI flags	`DNG_BLOCKS` 1, `LIGHTROOM` 2, `METADATA` 4, `COMPACT_RECORD` 8
C ABI field order	sha256, raw blocks, Lightroom, metadata digest, preserved, metadata record
Source	github.com/firstpair/phidentify
Sister projects	dng-compare, tiffgraph, Semantic Unison